

ОПИ: 80 вопросов с расширенными ответами

Материал собран по порядку тем из лекционного PDF и дополнен сведениями из предоставленных страниц Notion. Ответы оформлены как основа для устного ответа: определение, устройство или этапы, достоинства, ограничения и пример.

1. ISO/IEC 12207:2010: жизненный цикл ПО. Группы процессов ЖЦ

Жизненный цикл ПО — период от появления замысла программного продукта до прекращения его использования. ISO/IEC 12207 не навязывает одну модель разработки, а задаёт общий набор процессов, результатов и терминов, из которых организация формирует собственный процесс.

В системном контексте выделяются процессы соглашения, организационного обеспечения проекта, процессы проекта и технические процессы. Для программных средств отдельно рассматриваются реализация, поддержка и повторное использование. Соглашения регулируют отношения заказчика и поставщика; организационные процессы дают инфраструктуру и ресурсы; проектные отвечают за планирование и контроль; технические превращают требования в работающую систему и сопровождают её до вывода из эксплуатации. Стандарт необходимо адаптировать к масштабу, рискам и культуре проекта.

2. Модели ЖЦ: последовательная, инкрементная и эволюционная

Модель ЖЦ определяет порядок и взаимосвязь процессов разработки. В последовательной модели требования считаются известными заранее, а анализ, проектирование, реализация и тестирование выполняются по очереди. Она удобна для планирования, но плохо переносит поздние изменения.

В инкрементной модели целевой набор возможностей в основном известен, однако продукт поставляется частями: каждый инкремент добавляет законченную функциональность. В эволюционной модели уточняются не только реализации, но и сами требования — следующие версии формируются по обратной связи. Реальные проекты обычно сочетают оба подхода: функциональность поставляется инкрементами, а содержание будущих инкрементов эволюционирует.

3. Водопадная (каскадная) модель

Каскад состоит из последовательных стадий: требования к системе, требования к ПО, анализ, проектирование, кодирование, тестирование и эксплуатация. Результат каждой стадии документируется и становится входом следующей. Возврат формально возможен, но чем позже найден дефект, тем дороже переделка.

Плюсы модели — понятные контрольные точки, документы, бюджет и сроки. Она применима при устойчивых требованиях, дорогом аппаратном производстве или строгом регулировании. Главный минус — работающий продукт и обратная связь пользователя появляются поздно. Если на системном тестировании обнаружена ошибка требований, приходится менять документацию, архитектуру и код.

4. Методология Ройса

Уинстон Ройс описал каскадную схему и одновременно предложил меры против её риска. До полного анализа выполняется предварительный дизайн, чтобы проверить принципиальную реализуемость и критические решения. Ключевые результаты фиксируются в документах: требования, дизайн, план тестирования, руководство эксплуатации.

Ройс рекомендовал «сделать дважды»: сначала создать ограниченную пилотную версию, которая проверит основные характеристики. Тестирование следует планировать заранее, применять независимые инспекции, а пользователя подключать к предварительным, критическим и финальным обзорам. Поэтому его работа — не рекомендация слепого однонаправленного каскада, а попытка добавить обратную связь, прототипирование и управление риском.

5. Традиционная V-model Дж. Мансона и Б. Бёма

V-модель изгибает каскад так, чтобы каждой стадии спецификации соответствовал уровень проверки. В нижней точке находится кодирование. Модульное тестирование проверяет код и детальный дизайн, интеграционное — взаимодействие частей, системное — системную архитектуру, приёмочное — пользовательские требования, а эксплуатационная оценка — исходную концепцию.

Левая сторона преимущественно связана с верификацией: правильно ли строится продукт по спецификации. Правая сторона включает валидацию: тот ли продукт получает пользователь. Достоинство — тесты и критерии приёмки проектируются до кода и трассируются к требованиям. Ограничение — базовая схема остаётся последовательной и дорого реагирует на частые изменения.

6. Многопроходная (инкрементная) модель

Разработка делится на несколько проходов. Каждый проход включает уточнение части требований, детальное проектирование, кодирование, интеграцию и тестирование и заканчивается работающим инкрементом. Первый выпуск содержит наиболее важное ядро, последующие добавляют возможности.

Заказчик раньше видит результат, риски распределяются по итерациям, а финансирование можно связывать с поставками. Однако общая архитектура должна быть рассчитана на расширение, интерфейсы — стабильны, а регрессия — автоматизирована. Без этого быстрые инкременты приводят к архитектурной деградации, дорогому рефакторингу и рассинхронизации документации.

7. Модель прототипирования

Прототип — быстро созданная упрощённая модель интерфейса, бизнес-процесса или технически рискованной части. Пользователь оценивает её, после чего требования и решение уточняются. Выбрасываемый прототип служит только обучению и затем заменяется; эволюционный постепенно превращается в продукт.

Метод особенно полезен при неясных требованиях, новом интерфейсе или сомнительной технологии. Он снижает риск создать ненужную систему и помогает согласовать терминологию. Опасность состоит в том, что демонстрационный код с временными решениями могут объявить готовым: в нём обычно нет полноценной архитектуры, безопасности, обработки ошибок и эксплуатационных качеств.

8. RAD-методология

Rapid Application Development возникла как способ быстро создавать бизнес-приложения при помощи интегрированных сред, CASE-средств, генераторов экранов, готовых компонентов и средств работы с БД. После короткого планирования команда небольшими циклами создаёт формы и функции, а пользователь непрерывно участвует в оценке.

RAD сокращает время до полезного результата и хорошо работает для типовых информационных систем с доступным заказчиком. Его ограничения — зависимость от квалифицированной небольшой команды и пригодности готовых компонентов, а также риск слабой архитектуры и документации. Для высоконагруженной или критической системы RAD необходимо дополнять инженерией качества, безопасности и производительности.

9. Спиральная модель

Спиральная модель Б. Бёма строит проект как последовательность витков, каждый из которых ориентирован на риск. На витке определяют цели, альтернативы и ограничения; выявляют и уменьшают риски с помощью анализа или прототипа; разрабатывают и проверяют очередной результат; вместе с заказчиком планируют следующий виток.

Радиус спирали отражает накопленные затраты и степень готовности. Переход дальше оправдан, если критические риски достаточно снижены. Модель подходит большим инновационным системам, где ошибка архитектуры очень дорога. Для небольшого понятного приложения формальная оценка рисков и управление каждым витком могут оказаться избыточными.

10. UML: структурные и поведенческие диаграммы

UML — графический язык спецификации, визуализации и документирования системы. Структурные диаграммы описывают статическое устройство: классы, объекты, компоненты, пакеты и размещение. Например, диаграмма классов показывает сущности «Заказ», «Позиция», «Товар», их атрибуты и связи.

Поведенческие диаграммы показывают изменение во времени: use case, activity, state machine, sequence и другие interaction diagrams. Например, sequence-диаграмма отражает сообщения браузера, сервиса заказов и платёжной системы. UML — средство коммуникации, поэтому степень детализации выбирается по аудитории; диаграмма должна отвечать на конкретный вопрос, а не копировать весь код.

11. UML: Use-case модель

Use-case модель показывает внешние роли и цели, достигаемые ими через систему. Actor — внешняя роль человека, устройства, другой системы или времени; актор не является частью моделируемой системы. Use case формулируется как полезная цель, например «Оформить заказ», а system boundary отделяет внутреннюю функциональность.

Association показывает участие актора. **include** означает обязательное переиспользуемое поведение, **extend** — условное дополнение в точке расширения, generalization — специализацию актора или прецедента. Диаграмма даёт обзор требований, но не заменяет текст: основной сценарий, альтернативы, предусловия, постусловия и ошибки фиксируются отдельно.

12. UML: диаграмма классов

Класс изображается прямоугольником с именем, атрибутами и операциями; видимость обозначается **+**, **-**, **#**. Association показывает связь и кратность, aggregation — слабое отношение «целое–часть», composition — владение с общим жизненным циклом, generalization — наследование, realization — реализацию интерфейса, dependency — временную зависимость.

Пример: **Order** композиционно содержит **1..* OrderLine**, каждая строка ссылается на один **Product**, а **CardPayment** реализует **Payment**. Диаграмма может быть концептуальной моделью предметной области или проектной моделью кода. В первом случае не нужно механически переносить технические поля и методы.

13. UML: диаграмма последовательностей

Sequence diagram показывает взаимодействие участников во времени. Сверху размещаются акторы и объекты, вниз идут lifeline, горизонтальные стрелки обозначают синхронные, асинхронные сообщения и ответы. Activation показывает период выполнения. Фрагменты **alt**, **opt**, **loop**, **par** задают ветвление, необязательное действие, цикл и параллельность.

Для сценария оплаты можно показать Browser → OrderService → PaymentGateway → Database и отдельно альтернативу отказа банка. Диаграмма полезна для проектирования протокола и границ ответственности. Она не должна перечислять каждый внутренний вызов: важны сообщения, влияющие на архитектуру, состояние или контракт.

14. UML: диаграмма размещения

Deployment diagram описывает физическую архитектуру: устройства, серверы, VM, контейнеры, execution environments, развёрнутые artifacts и communication paths. На связях можно указывать протокол и порт. Например: браузер по HTTPS обращается к балансировщику, далее к двум контейнерам приложения, затем к PostgreSQL и Redis.

Диаграмма помогает обсуждать масштабирование, отказоустойчивость, сетевые границы, зоны доверия и размещение данных. В отличие от component diagram она отвечает не только «из каких программных частей состоит система», но и «где они исполняются и как физически связаны».

15. UP-методологии и основы RUP

Unified Process — семейство итеративно-инкрементных процессов, управляемых вариантами использования, архитектурой и рисками. Rational Unified Process конкретизирует этот подход ролями, деятельностью, артефактами, дисциплинами и рекомендациями.

Четыре фазы RUP: Inception, Elaboration, Construction, Transition. Каждая может содержать несколько итераций. Дисциплины требований, анализа и дизайна, реализации, тестирования, развёртывания, конфигурации и управления идут через весь ЖЦ, но с разной интенсивностью. Роль — набор обязанностей; деятельность создаёт или изменяет артефакт: модель, документ, код, тест или план.

16. RUP: фаза «Начало»

Inception определяет, зачем нужен проект и где его границы. Выявляются стейкхолдеры и ключевые use cases, создаются Vision и бизнес-кейс, оцениваются основные риски, ресурсы, сроки, стоимость и технологическая осуществимость.

Фаза завершается вехой Lifecycle Objectives. К этому моменту стороны должны одинаково понимать цели и объём, иметь первичный план и согласиться, что инвестиции оправданы. Это не полная спецификация: детализируются прежде всего функции и риски, влияющие на решение начинать проект.

17. RUP: фаза «Проектирование»

Elaboration должна превратить первоначальную идею в устойчивую архитектурную основу. Уточняются критичные требования, строится исполнимая базовая архитектура, прототипами снимаются главные технические и организационные риски, уточняются планы следующих итераций.

Веха Lifecycle Architecture означает, что архитектура выдержала проверку значимыми сценариями, основные риски снижены, оценки стоимости и срока стали правдоподобными, а критерии тестирования определены. Фаза не равна длительному написанию документов: её главный результат — доказанная архитектура, способная поддержать будущую реализацию.

18. RUP: фаза «Построение»

Construction превращает архитектурную основу в полноценный продукт. Итерации включают анализ оставшихся требований, реализацию, интеграцию и тестирование. Создаются рабочие выпуски, пользовательская документация, установочные средства, миграции и материалы обучения.

Задача — экономично реализовать требуемое качество, а не просто написать весь код. Веха Initial Operational Capability проверяет, достаточно ли стабилен выпуск для передачи пользователям, готовы ли инфраструктура и принимающая сторона, приемлемы ли затраты. Незакрытые критические дефекты или неготовое окружение означают, что переход преждевременен.

19. RUP: фаза «Внедрение»

Transition передаёт систему пользователям: развёртывание на целевой платформе, миграция данных, обучение, beta/приёмочные испытания, исправление дефектов, выпуск hotfix и настройка поддержки. Обратная связь используется для доведения продукта до эксплуатационной готовности.

Финальная веха Product Release отвечает на вопросы, удовлетворены ли пользователи, достигнута ли исходная концепция и приемлема ли итоговая стоимость. После выпуска работа может продолжаться в новом цикле развития; внедрение не означает прекращение сопровождения.

20. Манифест Agile (2001)

Манифест устанавливает четыре предпочтения: люди и взаимодействие важнее процессов и инструментов; работающий продукт важнее исчерпывающей документации; сотрудничество с заказчиком важнее согласования контракта; готовность к изменениям важнее следования исходному плану.

Элементы справа не объявляются бесполезными — им лишь придают меньший приоритет. Принципы Agile требуют ранней и регулярной поставки, устойчивого темпа, постоянной обратной связи, технического совершенства, простоты, самоорганизации и ретроспектив. Agile — не отсутствие планов и документов, а короткий цикл проверки предположений и способность менять план по фактам.

21. Scrum

Scrum — лёгкий фреймворк управления сложной разработкой. Product Owner отвечает за ценность и упорядочивает Product Backlog; Developers создают инкремент; Scrum Master помогает понимать Scrum и устранять системные препятствия. Артефакты: Product Backlog, Sprint Backlog и Increment; им соответствуют Product Goal, Sprint Goal и Definition of Done.

Спринт длится не более месяца, часто одну-две недели. Внутри проходят Sprint Planning, Daily Scrum, разработка, Sprint Review и Retrospective. Review проверяет продукт и адаптирует backlog, ретроспектива улучшает способ работы. Daily Scrum — синхронизация разработчиков вокруг цели спринта, а не отчёт руководителю.

22. Disciplined Agile 2.x

Disciplined Agile объединяет гибкие практики с дисциплиной UP/RUP и предлагает toolkit, а не один обязательный процесс. Команда выбирает жизненный цикл и способы работы с учётом размера, регуляtorики, распределённости, рисков и корпоративной среды. Часто используются фазы Inception, Construction и Transition, а внутри — Scrum, Kanban или непрерывная поставка.

DA охватывает не только командную разработку, но также архитектуру, данные, безопасность, DevOps, управление персоналом, портфелем и повторным использованием. Идея — оптимизировать весь поток ценности: локальная скорость одной команды бесполезна, если развёртывание, согласование или эксплуатация остаются узкими местами.

23. Требования и их иерархия

Иерархия связывает причину проекта с проверяемой реализацией: бизнес-цели → потребности стейкхолдеров → системные требования → требования к ПО и подсистемам → детальные спецификации и критерии приёмки. Верхние уровни отвечают «зачем», нижние — «что именно должно быть реализовано и как проверить».

Пример: «сократить оформление заказа на 30 %» → «покупатель оформляет заказ без регистрации» → «система создаёт заказ после успешной оплаты» → требования к API и интерфейсу → тест сценария и времени ответа. Трассируемость вверх показывает обоснование функции, вниз — покрывающие дизайн, код и тесты; она также помогает оценить влияние изменения.

24. Свойства и типы требований. FURPS+

Требование должно быть необходимым, однозначным, полным, непротиворечивым, реализуемым, проверяемым, трассируемым, приоритизированным и изменяемым. Формулировка описывает наблюдаемый результат и критерий, а не расплывчатое пожелание вроде «интерфейс должен быть быстрым».

FURPS классифицирует Functionality, Usability, Reliability, Performance и Supportability. «+» включает ограничения реализации, интерфейсы, платформу, законодательные и физические условия. Функциональные требования задают поведение, нефункциональные — качество и ограничения. Например, «найти товар» — функция, «p95 поиска не больше 800 мс при 100 RPS» — производительность.

25. Формулирование функциональных требований

Функциональное требование описывает наблюдаемую реакцию системы на действие или событие: кто инициирует, какие входы подаются, какие правила применяются и какой результат возникает. Хороший шаблон: «При условии X система должна выполнить Y и получить проверяемый результат Z».

Следует избегать слов «удобно», «быстро», «при необходимости», нескольких независимых требований в одном предложении и преждевременного технического решения. Требованиям назначают ID, источник, приоритет, обоснование, критерий приёма и связи. Пример: «После подтверждённой оплаты система в течение 5 секунд переводит заказ в Paid и публикует одно событие OrderPaid; повторный callback не создаёт второго события».

26. Требования к удобству использования и надёжности

Usability описывает эффективность, обучаемость, понятность, предотвращение ошибок, доступность и удовлетворённость конкретного пользователя в заданном контексте. Измеримый пример: «новый оператор после 15 минут обучения оформляет заявку за 3 минуты с не более чем одной исправляемой ошибкой».

Reliability включает вероятность корректной работы, availability, частоту отказов, MTBF, MTTR, сохранность и восстановление. Пример: «месячная доступность 99,9 %, RTO не более 30 минут, RPO не более 5 минут». Одного слова «надёжно» недостаточно: нужны период измерения, исключения, нагрузка и метод проверки.

27. Требования к производительности и поддерживаемости

Performance задают вместе со сценарием нагрузки: latency и его процентиля, throughput, число одновременных пользователей, объём данных, error rate и допустимые ресурсы. Пример: «при 500 пользователях p95 создания заказа ≤ 1 с, ошибки $< 0,1$ %, CPU ≤ 75 %».

Supportability охватывает анализируемость, модифицируемость, тестируемость, развёртываемость, локализацию, документацию и наблюдаемость. Примеры: изменение тарифа конфигурацией без перекомпиляции; диагностика ошибки по correlation ID за 15 минут; rollback релиза за 10 минут. Это полноценные требования к продукту, а не пожелания команде.

28. Атрибуты требований

У требования обычно есть уникальный идентификатор, название, текст, источник/владелец, обоснование, тип, приоритет, статус, версия, стабильность, риск, критерий приёма и связи трассируемости. Дополнительно фиксируют дату, автора, затронутый релиз и зависимости.

Атрибуты позволяют не только хранить текст, но и управлять портфелем требований: отделять согласованные от черновых, находить требования без тестов, планировать нестабильные области и оценивать изменение. Приоритет можно задавать MoSCoW или ценностью/стоимостью, но он должен быть согласован стейкхолдерами, а не назначен произвольно аналитиком.

29. Описание прецедента использования

Текстовый use case содержит название и цель, основного актора, заинтересованных лиц, trigger, предусловия, гарантии успеха и минимальные гарантии, основной успешный сценарий, альтернативные потоки, исключения и специальные нефункциональные требования.

Основной сценарий пишут как диалог «актор — система», где каждый шаг имеет наблюдаемый результат. Альтернативы ссылаются на конкретную точку основного потока. Например, при оплате основной поток заканчивается подтверждением заказа, а альтернативы описывают отказ банка, таймаут и повторный callback. Такой текст становится основой приёмочных тестов.

30. Риски и типы рисков

Риск — неопределённое событие, которое при наступлении влияет на цели. Он отличается от проблемы: проблема уже произошла. Риск описывают причиной, возможным событием, последствиями, вероятностью и влиянием; экспозицию часто приближённо оценивают как $\text{Probability} \times \text{Impact}$.

Выделяют проектные риски сроков, бюджета и персонала; технические — технологии, архитектуры, интеграции и производительности; продуктовые и качественные; бизнесовые — ценность, рынок, финансирование; внешние — законодательство, поставщики, инфраструктура и безопасность. Категории помогают не забыть источник, но управляют конкретным сценарием, а не абстрактной надписью «технический риск».

31. Управление рисками: деятельности оценки

Оценка начинается с идентификации через интервью, brainstorming, анализ WBS/архитектуры, опыт прошлых проектов и чек-листы. Риск формулируют «из-за причины может произойти событие, что приведёт к последствиям». Затем качественно оценивают вероятность, влияние на стоимость, срок, качество и репутацию, близость и обнаружимость.

Риски ранжируют в матрице или по экспозиции. Для дорогих решений применяют expected monetary value, деревья решений и Monte Carlo. Результат — реестр с владельцем, индикатором, сроком пересмотра и выбранной реакцией: избежать, снизить, передать или принять с резервом.

32. Управление рисками: контроль и управление

Контроль означает регулярный обзор реестра: не изменились ли вероятность и ущерб, сработали ли триггеры, эффективны ли меры, появились ли новые риски. Владелец выполняет действия снижения, а менеджер обеспечивает ресурсы и эскалацию. Значимые меры включаются в backlog и план, а не остаются заметкой.

Если событие наступило, риск становится issue: запускается contingency plan, корректируются бюджет, сроки и baseline, информируются стейкхолдеры. Для принятого риска заранее готовят резерв. После закрытия фиксируют lessons learned и остаточный риск. Цель управления — не устранить всю неопределённость, а принимать её осознанно.

33. Изменение и общая модель управления изменениями

Изменение начинается с Change Request: причина, инициатор, затронутые требования/версии, желаемый результат и приоритет. Далее запрос регистрируют, проверяют полноту, анализируют влияние на архитектуру, безопасность, сроки, стоимость, документацию и тесты, после чего владелец продукта или CCB принимает, отклоняет либо откладывает его.

Принятое изменение планируют, реализуют в VCS, проверяют review и тестами, включают в релиз и закрывают с обновлением связей. Baseline — утверждённое состояние конфигурации; его меняют контролируемо. Управление изменениями не запрещает изменения, а сохраняет согласованность и объясняет, кто, когда и почему изменил продукт.

34. Системы контроля версий. Одновременная модификация файлов

VCS хранит версии, авторов и историю, позволяет создавать ветви и теги, сравнивать и восстанавливать состояния. Современная модель `copy-modify-merge` разрешает нескольким разработчикам менять свои копии параллельно, а затем автоматически объединяет непересекающиеся правки.

Если правки затрагивают один участок, возникает конфликт, который решает человек, понимающий намерение обеих сторон. Практики: частые `update/pull`, короткие ветви, маленькие логические `commits`, `review`, CI и тест после `merge`. `Lock-modify-unlock` применяют для бинарных файлов, которые нельзя осмысленно слить, но блокировки уменьшают параллельность.

35. Subversion: архитектура системы и репозиторий

SVN — централизованная VCS. Сервер хранит единственный репозиторий и глобальные номера ревизий, клиент — рабочую копию с метаданными. Обычно каталоги `trunk`, `branches`, `tags` являются соглашением; дешёвое копирование внутри репозитория создаёт ветку или тег.

Доступ возможен через `https`, `svn`, `svn+ssh` или локальный `file`; сервером служит `svnserve` либо Apache с `mod_dav_svn`. Централизация упрощает единые права и резервное копирование, но `commit` и получение новых изменений зависят от сервера. Ревизия описывает атомарное изменение всего репозитория.

36. Subversion: основной цикл разработчика и команды

Цикл: `svn checkout URL` создаёт рабочую копию; после правок используют `svn status` и `svn diff`; `svn update` получает изменения коллег; затем выполняются сборка и тесты; `svn commit -m "..."` публикует атомарную ревизию.

Файлы добавляют и удаляют через `svn add`, `svn delete`, перемещают `svn move`; историю смотрят `svn log`, автора строки — `svn blame`, локальную правку отменяют `svn revert`. `svn copy` создаёт `branch/tag`, `svn switch` переводит рабочую копию, `svn merge` переносит диапазон изменений. Перед `commit` рабочая копия должна быть актуальна и проверена.

37. Subversion: конфликты и слияние изменений

Конфликт появляется при `update` или `merge`, когда SVN не может совместить правки. Для текста создаются конфликтные варианты и маркеры; для свойств и дерева возможны `property/tree conflicts`. Наличие чистого автоматического `merge` ещё не гарантирует логической корректности.

Разработчик изучает `base`, свою и входящую версии, формирует правильный итог, собирает и тестирует его, затем выполняет `svn resolve --accept working file` и `commit`. Ветку сливают выбранным диапазоном либо `reintegrate`-стратегией соответствующей версии SVN. Частые небольшие слияния и согласованные границы модулей снижают стоимость конфликтов.

38. Git: архитектура и команды

Git — распределённая VCS: каждый clone имеет полную локальную историю. Объекты — blob с содержимым, tree с каталогом, commit со снимком и родителями, annotated tag. Ветки и теги — ссылки на commits; HEAD показывает текущую ветку. Между рабочей директорией и репозиторием находится index/staging area.

Основные команды: `init`, `clone`, `status`, `add`, `commit`, `log`, `diff`, `branch`, `switch`, `merge`, `rebase`, `fetch`, `pull`, `push`, `tag`, `restore`, `revert`. `fetch` только получает ссылки, `pull` также интегрирует. `revert` создаёт обратный commit и безопасен для общей истории; переписывающий `reset/rebase` опасен для уже опубликованных commits.

39. Git: организация ветвей репозитория

Главная ветка должна быть защищена и проходить CI. Работа выполняется в коротких `feature/*` или `bugfix/*`, публикуется, проходит review и merge. Маленькие ветви и частая синхронизация уменьшают конфликт и время обратной связи. Незавершённую функциональность можно скрывать feature flags.

Стратегии различаются. Trunk-based development предполагает очень короткие ветви или прямую интеграцию в trunk и подходит непрерывной поставке. Release branches полезны при параллельной поддержке версий. Долгоживущие ветви требуют регулярного merge и часто создают «интеграцию в конце».

40. Плагин git-flow

Классический git-flow использует `main` для выпущенных версий и `develop` для интеграции. `feature/*` ответвляются от develop, `release/*` стабилизируют выпуск и сливаются в main/develop, `hotfix/*` начинаются от main и срочно исправляют production. Релизы помечаются tags.

Команды расширения автоматизируют создание и завершение ветвей: `git flow init`, `feature start/finish`, `release start/finish`, `hotfix start/finish`. Схема удобна при редких версионных релизах и нескольких поддерживаемых версиях. Для частого CD она тяжела: две постоянные ветви и позднее слияние увеличивают задержку, поэтому часто выбирают упрощённый GitHub Flow или trunk-based.

41. Системы автоматической сборки: предпосылки появления

Ручная сборка требует помнить порядок компиляции, параметры, версии библиотек, тесты и упаковку. С ростом проекта результат начинает зависеть от машины и знаний человека — возникает проблема «у меня собирается». Система сборки хранит процесс в версионизируемом описании, строит граф целей и зависимостей, выполняет только нужные шаги и создаёт воспроизводимый артефакт. Она даёт единые команды разработчику и CI. Для настоящей воспроизводимости фиксируют не только исходники, но также версии инструментов, зависимостей и окружение.

42. Make и Makefile

Make представляет сборку как граф правил: цель, её зависимости и рецепт. Если цели нет или она старше зависимости, рецепт выполняется, поэтому после изменения файла перекомпилируется только затронутая часть. В Makefile применяют переменные, pattern rules и `$(цель)`, `$(<)` (первая зависимость), `$(^)` (все зависимости). `.PHONY` помечает команды вроде `clean`. Make прост и универсален, но рецепты зависят от shell/ОС, а программист обязан точно описать зависимости.

43. Ant и команды Ant

Apache Ant — XML-система сборки Java. В `build.xml` описываются project, properties, targets и tasks; `depends` задаёт граф целей. Задачи `javac`, `junit`, `jar`, `copy`, `delete` выполняют типовые операции. Команды: `ant` запускает default target; `ant clean, compile, test, jar` — именованные цели; `ant -Dname=value target` передаёт property; `ant -p` выводит цели. Ant гибок, но XML многословен, стандартного ЖЦ и современного встроенного управления зависимостями нет.

44. Ant-Ivy

Apache Ivy добавляет к Ant разрешение зависимостей. В `ivy.xml` задаются модуль, версия, конфигурации и требуемые библиотеки. `ivy:resolve` строит транзитивный граф и загружает артефакты, `ivy:retrieve` раскладывает их локально, `ivy:classpath` создаёт classpath. Ivy работает с Maven-репозиториями и собственными resolvers. Связка сохраняет гибкость Ant, но структуру проекта и жизненный цикл команда определяет сама, тогда как Maven предлагает соглашения из коробки.

45. Maven: POM, репозитории и зависимости

`pom.xml` содержит координаты, parent, properties, packaging, modules, dependencies, build/plugins, profiles и repositories. Зависимость задаётся GAV и scope; транзитивные зависимости Maven получает автоматически. Ненужные исключают через `exclusions`, версии централизуют в `dependencyManagement/BOM`. Локальный репозиторий — кэш пользователя; удалённые — Central и корпоративный Nexus/Artifactory. Корпоративный проху повышает воспроизводимость и позволяет контролировать лицензии и уязвимости.

46. Maven: структура проекта и GAV

Соглашение Maven: `src/main/java`, `src/main/resources`, `src/test/java`, `src/test/resources`, `pom.xml`; результат — в `target`. Благодаря этому типовой проект почти не требует процедурного скрипта. GAV — `groupId:artifactId:version`: глобально однозначные координаты артефакта; packaging добавляет тип `jar/war/pom`. Многомодульный parent POM перечисляет модули и централизует настройки. Координаты позволяют найти бинарный артефакт и его метаданные в любом совместимом репозитории.

47. Maven: зависимости, жизненный цикл и плагины

Основной lifecycle: `validate` → `compile` → `test` → `package` → `verify` → `install` → `deploy`; поздняя фаза включает предыдущие. Фазы исполняются goals плагинов: `compiler`, `surefire/failsafe`, `jar/war`, `install`, `deploy`. Scopes `compile`, `provided`, `runtime`, `test` определяют classpath и транзитивность. Конфликт версий проверяют `mvn dependency:tree`. Для CI типична команда `mvn clean verify; install` кладёт пакет локально, `deploy` публикует удалённо.

48. Gradle: преимущества и файл сборки

Gradle строит граф задач; `build.gradle` использует Groovy, `build.gradle.kts` — Kotlin. В скрипте объявляют plugins, repositories, dependencies, tasks; Wrapper фиксирует версию Gradle. Сильные стороны: инкрементальность по входам/выходам, build cache, daemon, параллельность и гибкие плагины. По сравнению с Maven меньше жёстких соглашений, но эта свобода требует дисциплины: скрытые side effects делают кэш и сборку невоспроизводимыми.

49. GNU autotools: создание конфигурации

Autotools обеспечивает переносимую сборку C/C++ в Unix. `configure.ac` задаёт метаданные и проверки компилятора, headers, функций и библиотек; `Makefile.am` описывает программы, исходники и тесты; Libtool помогает собирать библиотеки. `autoreconf -i` создаёт `configure`, `Makefile.in` и служебные файлы. В исходный дистрибутив обычно включают готовый `configure`, чтобы пользователю не требовались Autoconf/Automake. Лучше проверять наличие возможности, чем угадывать её по названию ОС.

50. GNU autotools: конфигурация и сборка

Пользователь выполняет `./configure --prefix=...`: скрипт проверяет окружение и создаёт `Makefile/config.h`. Далее `make` собирает, `make check` тестирует, `make install` устанавливает. `make clean` удаляет результаты, `make distclean` — также конфигурацию. Out-of-tree build сохраняет исходники чистыми; причины неудачных проверок находятся в `config.log`. Для сопровождающего `make dist` создаёт пакет, а `make distcheck` проверяет его сборку, тесты и удаление.

51. Серверы сборки и непрерывная интеграция

CI-сервер на commit/PR получает чистую копию, восстанавливает зависимости, собирает, запускает анализ и тесты, публикует отчёты и неизменяемый артефакт. Примеры: Jenkins, GitLab CI, GitHub Actions, TeamCity. Принципы: частая интеграция, быстрая самопроверяемая сборка, видимость результата и немедленное исправление красного pipeline. Быстрые проверки выполняют раньше дорогих. Артефакт собирают один раз и продвигают между средами, иначе production-сборка может отличаться.

52. Основные понятия и цели тестирования

Ошибка человека создаёт defect в артефакте; при исполнении defect может вызвать failure. Тестирование включает статическую оценку и динамическое выполнение для обнаружения дефектов, проверки требований и предоставления информации о риске. Debugging локализует и исправляет уже обнаруженную причину. Цели тестирования: предотвращать дефекты, находить ошибки, подтверждать соответствие и поддерживать решение о выпуске. Оно снижает риск, но не доказывает отсутствие ошибок.

53. Полное покрытие и его достижимость

Покрываемость бывает по требованиям, строкам, ветвям, условиям, путям, состояниям и комбинациям данных; «100 %» без критерия бессмысленно. Полное покрытие путей обычно невозможно: n независимых ветвлений даёт до 2^n путей, а цикл — потенциально бесконечное число. Поэтому выбирают риск-ориентированную комбинацию требований, ветвей, эквивалентных классов, границ и мутаций. Метрика помогает искать пробелы, но высокий процент сам по себе не гарантирует хорошие проверки.

54. Статическое и динамическое тестирование

Статическое тестирование не исполняет программу: review требований/архитектуры/кода, inspection, компилятор, lint, SAST, анализ типов и data flow. Оно рано находит неоднозначности, dead code и потенциальные уязвимости. Динамическое запускает код и сравнивает результат с oracle: unit,

integration, system, acceptance, performance. Подходы дополняют друг друга: статика указывает подозрительный путь, динамический тест подтверждает реальное поведение и временные свойства.

55. Автоматизация и ручное тестирование

Автотесты повторяемы и быстры, полезны для unit/API/контрактов и стабильной регрессии, но требуют разработки и поддержки. Хрупкий UI-тест, падающий без дефекта, разрушает доверие. Ручное тестирование нужно для exploratory testing, usability, визуальной оценки и новой функции.

Рациональная пирамида: много быстрых unit, меньше integration/API, мало дорогих E2E плюс исследовательские сессии. Автоматизируют частые проверки с однозначным ожидаемым результатом.

56. Источники данных, роли и деятельности

Данные получают из синтетических генераторов, fixtures, обезличенной production-выборки, property-based генерации, mocks/service virtualization. Нужны нормальные, граничные и ошибочные случаи; данные должны быть воспроизводимыми, изолированными и законными. Заказчик задаёт приёмку, разработчик отвечает за unit и исправления, QA анализирует риск и проектирует проверки, lead формирует стратегию, DevOps поддерживает среды. Деятельности: план, анализ, дизайн, данные/среда, выполнение, дефекты, отчёт и улучшение.

57. Тестовый случай и сценарий

Сценарий высокоуровнево описывает поток, например «покупка зарегистрированным пользователем». Test case содержит ID, требование, предусловия, данные, шаги, ожидаемые результаты, фактический результат и статус. Хороший тест имеет ясный oracle, независим и повторяем; cleanup возвращает среду в известное состояние. Один сценарий порождает случаи успешной оплаты, отказа, таймаута, повторного callback, нехватки товара и граничной суммы.

58. Выбор покрытия и числа тестов. Эквивалентность

Число тестов зависит от риска, критичности, сложности, частоты, изменчивости, истории дефектов и стоимости. Эквивалентное разбиение делит входы на классы с ожидаемо одинаковым поведением; берут представителя каждого валидного и невалидного класса. Затем проверяют границы. Для возраста 18–65 классы: `<18`, `18..65`, `>65`; сильные границы: 17, 18, 19, 64, 65, 66. Комбинаторный набор сокращают pairwise или риск-анализом.

59. Модульное тестирование и JUnit 4

Unit test проверяет небольшую единицу поведения быстро и изолированно; БД, сеть и часы заменяются stubs/mocks/fakes. Структура — Arrange–Act–Assert; проверять лучше публичный результат, а не внутренние вызовы. JUnit 4 использует `@Test`, `@Before/@After`, `@BeforeClass/@AfterClass`, `@Ignore`, assertions и `expected`. Тесты не должны зависеть от порядка или общего состояния; Maven/Gradle и CI запускают весь suite при каждом изменении.

60. Интеграционное тестирование и стратегии

Интеграционные тесты проверяют контракты модулей, сервисов, БД, очередей и API: форматы, транзакции, ошибки, retry и timeout. Big bang объединяет всё сразу и плохо локализует дефект. Top-down заменяет нижние части stubs, bottom-up использует drivers, sandwich сочетает подходы;

вертикальные срезы рано дают end-to-end ценность. Частая небольшая интеграция и contract tests безопаснее позднего большого объединения.

61. Функциональное тестирование и Selenium

Функциональная проверка сопоставляет наблюдаемое поведение с требованием через API, UI или систему, включая позитивные, негативные и граничные случаи. Selenium WebDriver управляет браузером: открыть страницу, найти элемент устойчивым locator, выполнить действие, явно дождаться состояния и проверить assertion. Следует избегать `sleep`, координат и хрупких XPath. UI E2E оставляют для критичных путей, а основную логику проверяют быстрее на unit/API-уровне.

62. Статические техники и анализ кода

Неформальный review не имеет строгой процедуры; walkthrough ведёт автор; technical review проводят специалисты; inspection формализует роли, подготовку, регистрацию дефектов и follow-up. Автоматический анализ исследует AST/байткод: стиль, сложность, типы, data/control flow, утечки, инъекции и уязвимые зависимости. Инструменты встраивают в IDE/CI с quality gate. Срабатывания требуют triage: ложные подавляют с обоснованием, а не отключают правила ради зелёного отчёта.

63. Системное и производительности тестирование

System test проверяет полностью интегрированный продукт в близком к боевому окружении: функции, безопасность, совместимость, восстановление, установку и эксплуатацию. Performance testing включает baseline, load, stress, spike, endurance и volume. Заранее задаются workload и SLA; после warm-up измеряют p50/p95/p99 latency, throughput, errors и saturation CPU/RAM/disk/network. Генератор не должен сам ограничивать результат, а тест должен повторяться после изменения.

64. Alpha-, beta- и приёмочное тестирование

Alpha проводится разработчиком в контролируемой среде и устраняет крупные дефекты до внешнего доступа. Beta отдаёт почти готовый продукт ограниченной группе реальных пользователей в их окружении и выявляет эксплуатационные/совместимые проблемы. Приёмочное тестирование выполняет заказчик по согласованным критериям и принимает бизнес-решение о поставке; оно бывает пользовательским, контрактным, регуляторным и операционным. Alpha/beta дают обратную связь, а приёмка связана с обязательствами.

65. Аспекты быстродействия и влияние измерения

Быстродействие включает latency, throughput, CPU, ожидание I/O/lock, память и масштабируемость при конкретной нагрузке. Важны распределение и процентиля, а не одно среднее. Измеритель меняет систему: instrumentation добавляет вызовы, tracing — I/O, sampling прерывает потоки, debugger меняет планирование/cache. Поэтому начинают с дешёвых метрик, затем локально включают точную трассу, оценивают overhead и подтверждают результат повторным чистым прогоном.

66. Ключевые характеристики производительности

Latency — длительность операции, response time включает очередь; throughput — операций за время; utilization — занятость; saturation — накопленная очередь; concurrency — параллельная работа; error rate — доля отказов. До насыщения throughput растёт, а возле предела очередь и хвостовая задержка

резко увеличиваются. SLA задаёт метрики совместно: например, при 300 RPS p95 < 500 мс, ошибки < 0,1 %, CPU < 75 %.

67. Нисходящий поиск узких мест

Сначала фиксируют пользовательский симптом, workload и нарушенный SLO. Затем по метрикам системы определяют класс ограничения: CPU, память, диск, сеть, внешняя зависимость или lock. Далее сужают поиск до хоста, процесса, потока, запроса и функции. Формулируют одну гипотезу, меняют один фактор и повторяют тот же тест. Движение от системы к коду защищает от оптимизации красивого, но незначимого участка.

68. Пирамида памяти

Регистры → L1/L2/L3 → RAM → SSD/HDD → сеть: нижние уровни больше и дешевле, но медленнее. CPU получает cache lines, поэтому temporal/spatial locality критична. Компактные структуры, последовательный обход и обработка блоками повышают hit rate; случайный доступ, pointer chasing, false sharing и рабочий набор больше RAM вызывают misses, page faults и I/O. Из-за этого алгоритм с лучшей теоретической сложностью иногда проигрывает более cache-friendly реализации.

69. Мониторинг процессов

Смотрят CPU user/system, runnable/blocked threads, context switches, RSS/VSZ, allocation/GC, page faults, descriptors, network/disk I/O. Linux: **ps**, **top**, **pidstat**, **iostat**, **lsof**, **strace**, **perf**; Windows: Task Manager, Resource Monitor, PerfMon, Process Explorer, ETW. Метрики обязательно связывают с временем запроса, нагрузкой, версией и baseline: один снимок без контекста редко объясняет причину.

70. Мониторинг виртуальной памяти

VSZ показывает зарезервированное адресное пространство, RSS — резидентные страницы; большой VSZ не обязательно утечка. Minor fault обслуживается без диска, major fault требует чтения. Наблюдают available memory, cache, swap, page-in/out, working set и OOM. Linux: **free**, **vmstat**, **sar -B**, **smaps**; Windows: Commit, Working Set, Hard Faults/sec. Постоянный swap-in/out и малая полезная работа означают thrashing.

71. Буферизированный файловый I/O

Пользовательский буфер и page cache объединяют мелкие операции, сокращая syscalls и физический I/O. Завершение buffered write обычно означает запись в память, не на носитель; durability требует **fsync**/flush и корректного протокола. Мониторят throughput, IOPS, latency, queue depth, utilization, cache hit, dirty pages и writeback. Direct I/O не всегда быстрее: оно обходит cache, но перекладывает выравнивание и управление буферами на приложение.

72. Мониторинг Windows и Linux

Windows: Task Manager, Resource Monitor, Performance Monitor, ETW/WPR/WPA и Process Explorer. Linux: **uptime**, **top**, **vmstat**, **mpstat**, **pidstat**, **iostat**, **sar**, **free**, **ss**, **perf**. Метод един: определить workload/ время, посмотреть latency, traffic, errors и saturation, найти насыщенный ресурс, затем процесс и операцию. Для истории нужны централизованные метрики, логи, traces и алерты по SLO.

73. Linux «за 60 секунд»

`uptime` показывает load; `dmesg | tail` — ошибки ядра; `vmstat 1` — очередь, CPU, swap и I/O; `mpstat -P ALL 1` — ядра; `pidstat 1` — процессы; `iostat -xz 1` — диски; `free -m` — память; `sar -n DEV 1` — сеть; затем `top`. Цель минуты — классифицировать проблему и выбрать следующую проверку, а не поставить окончательный диагноз. Команды запускают во время симптома и сравнивают с baseline.

74. Тестовая нагрузка и нагрузчики

Workload строят по production-статистике: сценарии, доли операций, think time, arrival rate/concurrency, размеры данных, cache hit, длительность и рост. Заранее задают SLO, warm-up и критерий остановки. Инструменты: JMeter, Gatling, k6, Locust; wrk/ab для простого HTTP; Selenium — лишь для небольшого числа browser flows. Генератор контролируют отдельно: его CPU, сеть или порты не должны стать ложным пределом сервера.

75. Профилирование приложений

Sampling периодически снимает стеки и с малым overhead показывает горячие функции. Instrumentation считает вызовы точнее, но сильнее искажает работу. Tracing связывает части распределённого запроса; allocation/heap profiler исследует память/GC. CPU time отличается от wall time: операция может ждать сеть или lock. Профиль снимают под репрезентативной нагрузкой, сопоставляют с системными метриками и подтверждают гипотезу повторным тестом.

76. Компромиссы производительности

Кэш меняет latency на память и сложность invalidation; параллелизм повышает throughput, но добавляет locks/context switches; batching уменьшает overhead, увеличивая ожидание; сжатие экономит сеть ценой CPU. Индекс ускоряет чтение, но замедляет запись и занимает место; replication повышает доступность, но создаёт lag. Оптимизацию выбирают по SLO и цене ресурса, а после изменения проверяют возможную деградацию других качеств.

77. Высокий процент SYS

Высокий `%sys` означает работу ядра: частые syscalls, I/O, interrupts, context switches, page faults, contention или драйвер. Диагностика: `vmstat`, `pidstat -w`, `strace -c`, `perf`, сетевые/дисковые счётчики. Меры по причине: batching/буферизация, меньше копирования и пакетов, reuse соединений, меньше потоков, исправление swap/lock или драйвера. Добавление CPU может лишь скрыть патологический паттерн.

78. Высокий процент IO wait

`%iowait` — простой CPU в ожидании I/O; трактовка зависит от ОС и ядер. Через `iostat -xz`, `pidstat -d`, `iostat`, latency/queue находят устройство и процесс, проверяют major faults/swap. Затем уменьшают операции и объём, применяют индексы, кэш, batching, последовательный/async I/O, распределяют данные или ускоряют носитель. Если причина в RAM, устраняют утечку/рабочий набор. Цель — улучшить latency, а не косметически снизить процент.

79. Высокий процент Idle

Высокий %idle нормален, если SLA выполняется. Если система медленна, а CPU свободен, она ждёт сеть, диск, lock, внешний сервис, connection pool или очередь. Проверяют состояния потоков, off-CPU profile, iowait, run queue и таймауты. Возможен случай, когда одно ядро насыщено, а остальные idle из-за серийности. Решение — убрать ожидание или распараллелить безопасный участок, а не искусственно загружать CPU.

80. Высокий процент User

Высокий %user означает исполнение пользовательского кода. Проверяют полезный throughput и загрузку ядер, затем снимают sampling profile/flame graph. Причины: плохая сложность, сериализация, regex, копирование, аллокации/GC, busy-wait. Улучшают алгоритм и структуру данных, удаляют ненужную работу, кэшируют повторный результат, уменьшают аллокации; потом распараллеливают и масштабируют. Если код разумен, высокая загрузка может быть нормальной полезной работой.

Для устного ответа используйте схему: **определение** → **элементы/этапы** → **применение** → **достоинства и ограничения** → **пример**. Для требований называйте метрику, для тестов — уровень и oracle, для производительности — workload, измерение, гипотезу и повторный тест.